

M431 & M122

Projektarbeit



1.1 Inhaltsverzeichnis

1.0	Titelblatt
1.1	Inhaltsverzeichnis
2.0	Dokumentation aller IPERKA-Phasen
2.1	Informieren
2.2	Planen
2.3	Entscheiden
2.4	Realisieren
2.5	Kontrollieren
2.6	Auswerten
3.0	Quellenverzeichnis

2.0 Dokumentation aller IPERKA-Phasen

2.1 Informieren

Am Anfang dieses eigenständigen Projekts stand ich erst einmal vor einer klassischen Herausforderung: Welches Thema bietet genügend Tiefgang, ist aber gleichzeitig auch im Alltag wirklich nützlich? Die Initialzündung kam schliesslich durch den Stundenplan. Da wir parallel zum Modul M431 auch im Modul M122 ein neues Projekt gestartet haben, lag es für mich nahe, diese beiden Welten miteinander zu verknüpfen. Die Projektidee davon ist es ein Hilfreiches Automatisierungstool für die Open Source Plattform Home Assistant zu erstellen welches Computerdaten hochlädt und Befehle empfangen kann. Dieses Projekt richtet sich im Grunde an alle Technologie-Begeisterten, die bereits Home Assistant nutzen und ihren Computer endlich intelligent in ihr bestehendes Smart Home System einbinden möchten. Die Möglichkeiten, die sich daraus ergeben, sind riesig. Ein klassisches Beispiel aus dem Alltag: Der Akku des Laptops neigt sich dem Ende zu und sinkt unter 25 Prozent. Das PowerShell-Skript registriert das sofort, meldet es an Home Assistant, und prompt schaltet sich die smarte Stehlampe im Zimmer rot ein so übersieht man garantiert nie wieder, das Ladekabel einzustecken.

Ich führe das Projekt allein durch. Von der ersten Konzeption über die Skripterstellung bis hin zur Testphase liegt die gesamte Umsetzung in meiner Hand, was mir die Freiheit gibt, das Tool genau nach meinen Vorstellungen zu gestalten.

2.2 Planen

Eine meiner wichtigsten Entscheidungen betrifft die Plattformkompatibilität. Das finale Produkt soll plattformübergreifend einsetzbar sein und sowohl für Windows als auch für Linux-Distributionen zur Verfügung stehen. Das bedeutet, dass die Kernfunktionen so konzipiert werden müssen, dass sie auf beiden Systemen reibungslos laufen, sei es über PowerShell-Skripte unter Windows oder entsprechende Gegenstücke in der Linux Experience. Um den Überblick über den aktuellen Entwicklungsstand nicht zu verlieren, habe ich eine detaillierte Checkliste für die verschiedenen

Features		
Metric / Action	Linux (Not Supported)	Windows
CPU usage (%)	✓	✓
Memory usage (%)	✓	✓
Disk usage (%)	✓	✓
Battery level & status	✓	✓
System uptime	✓	✓
IP address	✓	✓
Online / Offline status	✓	✓
Remote: Shutdown	✓	✓
Remote: Reboot	✓	✓
Remote: Sleep / Hibernate	✓	✓
Remote: Lock screen	✓	✓
Auto-start on login	systemd user service	Windows Autostart shortcut
Auto device registration	✓	✓

Funktionen ausgearbeitet. Diese Feature-Liste ist jedoch weit mehr als nur eine einfache To-do-Liste. Jede einzelne Funktion, die erfolgreich implementiert und getestet wurde, dient im Projektverlauf sogleich als wichtiger Meilenstein. Auf diese Weise lässt sich der tatsächliche Baufortschritt jederzeit messbar überprüfen, und die Meilensteine bieten mir eine verlässliche Zeitorientierung, um zu sehen, ob ich im geplanten Zeitrahmen liege.

2.3 Entscheiden

Nachdem bei mir die funktionalen Anforderungen und die theoretische Plattformkompatibilität feststanden, ging ich im nächsten Schritt in die konkrete Umsetzung und die Evaluation der verschiedenen Systemvarianten. Ein zentraler Entscheidungsfaktor war hierbei die Frage, wie intensiv die jeweiligen Plattformen nach dem Release von mir betreut und weiterentwickelt werden sollen. Folgende Lösungen habe ich in Betracht gezogen:

Eine reine Windows-Lösung: Fokus ausschliesslich auf PowerShell und die Windows-Infrastruktur.

Eine reine Linux-Lösung: Umsetzung basierend auf Shell-Skripten (Bash) für Linux-Systeme.

Eine Cross Plattform Lösung: Entwicklung für beide Betriebssysteme, um eine maximale Flexibilität zu gewährleisten.

Kriterium	Variante 1: Windows	Variante 2: Linux	Variante 3: Hybrid Windows & Linux
Entwicklungsaufwand	5	4	2
Persönlicher Nutzen	5	1	5
Funktionsumfang	3	3	5
Lerneffekt	3	3	5
Gesamtergebnis	16	11	17

Die Nutzwertanalyse zeigt ein knappes, aber klares Ergebnis: Die hybride Variante 3 schneidet durch den enormen Lerneffekt und den maximalen Funktionsumfang am besten ab.

2.4 Realisieren

Wie geplant, lag der Hauptfokus auf der Entwicklung für das Windows-Betriebssystem. Als fundamentale Basis und primäre Programmierumgebung habe ich hierfür PowerShell gewählt. Mit PowerShell lassen sich tiefe Systemeigenschaften eines Windows-PCs besonders elegant und zuverlässig auslesen sowie administrative Befehle direkt ausführen. Ebenfalls war PowerShell teil des Moduls 122 bei Christian. Für die Linux-Kompatibilität kam ein moderner Entwicklungsansatz zum Zuge: Um das Skript effizient in eine Linux-Umgebung zu portieren, habe ich den bestehenden PowerShell Code mithilfe des KI-Entwicklungstools Claude Code analysiert und präzise in ein Bash Skript umgeschrieben. Um das Projekt professionell zu verwalten, die Versionskontrolle zu sichern und das Ganze für die Community zugänglich zu machen, wurde der gesamte Quellcode auf GitHub hochgeladen. Das Repository dient jedoch nicht nur zur reinen Veranschaulichung und als transparente Projektdokumentation; es wurde auch so strukturiert, dass technikaffine Nutzer das Automatisierungstool direkt über HACS Home Assistant Community Store als benutzerdefiniertes Repository in ihr bestehendes Smart Home einbinden und bequem herunterladen können. Der vollständige Code sowie alle Installationsanweisungen sind unter folgendem Link öffentlich einsehbar:

[GitHub Repository: Home Assistant PC Sync](#)

Abgerundet wird die Realisierungsphase durch ein praktisches Hilfsmittel für Endanwender: Da eine reine Code-Dokumentation oft trocken ist, habe ich zusätzlich ein kurzes, anschauliches Dokumentationsvideo erstellt. Dieses Video wurde auch ein wenig als Promotion / Werbung von mir gedreht mit Epischer Musik zwischen den Segmenten.

Dokumentationsvideo zum Projekt:

<https://immich.benjamink.ch/s/122projekt>

2.5 Kontrollieren

Eine sorgfältige Qualitätskontrolle war während des gesamten Projektverlaufs ein entscheidender Faktor, um die Stabilität und Zuverlässigkeit des Synchronisierungstools sicherzustellen. Ich habe mich hierbei nicht nur auf einen finalen Test verlassen, sondern den aktuellen Entwicklungsstand im Sinne eines kontinuierlichen Verbesserungsprozesses immer wieder fortlaufend kontrolliert und auf Herz und Nieren geprüft. Folgendes habe ich dabei kontrolliert:

Code-Review und Optimierung: Nach der erfolgreichen Implementierung der einzelnen Funktionen habe ich meine Aufgaben systematisch nachkontrolliert. Dabei wurde der geschriebene PowerShell und Bash Code nochmals kritisch analysiert.

Funktionstests der Features: Zum Abschluss der Entwicklungsphase folgte ein umfassender End-to-End-Test. Ich habe sämtliche im Tool integrierten Funktionen und virtuellen «Knöpfe» in Home Assistant einzeln durchgetestet.

Medien- und Abgabekontrolle: Auch die begleitenden Projektmaterialien wurden einer finalen Durchsicht unterzogen. Bevor ich das Projekt endgültig abgegeben habe, habe ich mir das erstellte Dokumentationsvideo noch einmal ganz von Anfang bis Ende angesehen. Damit stellte ich sicher, dass die Erklärungen verständlich sind, die Bild- und Tonqualität stimmt und der Ablauf für aussenstehende Betrachter logisch nachvollziehbar ist.

Das Fazit der Kontrollphase fällt durchwegs positiv aus: Wie bereits im vorherigen Abschnitt «2.2 Planen» anhand der Feature-Matrix aufgezeigt, konnten sämtliche gesteckten Ziele und Meilensteine ohne Abstriche erreicht werden.

2.6 Auswerten

Rückblickend kann ich sagen, dass mir die Arbeit an diesem Vorhaben extrem viel Spass gemacht hat. Es war unglaublich motivierend, ein Projekt von Grund auf selbst zu planen, zu entwickeln und am Ende ein funktionierendes Produkt in den Händen zu halten (Naja mehr oder weniger in den Händen halten aber ich glaube der Punkt ist klar). Der grösste Erfolg liegt für mich im praktischen Nutzen: Im Gegensatz zu rein theoretischen Schulaufgaben habe ich nun ein massgeschneidertes Tool auf GitHub und HACS veröffentlicht, welches ich ab jetzt auch im Alltag tagtäglich selbst aktiv gebrauchen und in mein eigenes Smart Home integrieren kann. Zu sehen, wie der eigene Computer nahtlos mit Home Assistant kommuniziert und Befehle ausführt, ist extrem unkompliziert und ein echter Gewinn für meine persönliche Heimautomatisierung.

Neben dem fertigen Tool nehme ich vor allem viele wertvolle Erkenntnisse und positive Learnings aus dieser Projektphase mit:

Vertiefung der Skriptsprachen: Durch die intensive Arbeit mit PowerShell konnte ich meine Fähigkeiten in der Windows-Systemadministration deutlich ausbauen. Ich verstehe nun viel besser, wie man tiefere Betriebssystemdaten ausliest und verarbeitet.

Effizienter Einsatz von KI-Tools: Die Zusammenarbeit mit *Claude Code* beim Umschreiben des Codes in ein Linux-Bash-Skript war eine extrem lehrreiche Erfahrung. Es hat mir gezeigt, wie moderne KI-Werkzeuge den Entwicklungsprozess beschleunigen und die Brücke zwischen verschiedenen Plattformen schlagen können, ohne dass man das Rad jedes Mal neu erfinden muss.

Projektmanagement und Open Source: Von der ersten Idee über die strukturierte Nutzwertanalyse bis hin zur Veröffentlichung auf GitHub inklusive Dokumentationsvideo habe ich den gesamten Lebenszyklus einer Software-Entwicklung durchlaufen. Auch das Einrichten eines HACS-kompatiblen Repositories war Neuland für mich und ein wichtiges Learning im Bereich Open-Source-Distribution.

3.0 Quellenverzeichnis

Logo PowerShell : <https://en.wikipedia.org/wiki/PowerShell>

Logo Home Assistant : https://en.wikipedia.org/wiki/Home_Assistant

HACS : <https://www.hacs.xyz/>

Videovorführung und Werbung für das Projekt:

<https://immich.benjamink.ch/s/122projekt>

Github: <https://github.com/RubeldiRubelda/Home-Assistant-PC-Sync/tree/main>

Projektrelease: <https://github.com/RubeldiRubelda/Home-Assistant-PC-Sync/releases/tag/v1.0.0>

Recherchen zu PowerShell: <https://infmod.gitlab.io/m122/>

KI-Transparenzhinweis:  Gemini

Zur Korrektur der Projektarbeit wurde Gemini 3.5 Flash am 13.06.2026 angewendet. Folgender Prompt wurde genutzt: Folgende Texte sind für CH-GRammatik zu korrigieren überarbeite die Texte der Grammatik nach und ändere schlechte Textpassagen für besseren Lesefluss ab.